

SPI Interface Project
“SPI Slave with Single Port Ram”

Verilog implementation and FPGA flow

BY: Youssif Ahmed Sayed

Table of Contents

1. Project Overview	4
1.1. Introduction.....	4
1.2. SPI Interface Diagram.....	4
1.3. Project Specifications.....	4
1.3.1. SPI Slave Interface.....	4
1.3.2. Single Port Ram	5
1.3.3. SPI Wrapper.....	6
1.4. Design Flow	7
1.5. Summary	7
3. RTL Design.....	8
3.1. Single Port Ram module	8
3.2. SPI Slave Interface “Sequential Encoding”	9
3.3. SPI Wrapper module.....	11
4. SPI Wrapper Testbench as Master	12
5. Do file to run testbench.....	14
6. Snippets from the waveforms	15
7. Constrains File after adding a debug core “sequential encoding”	16
8. Elaboration Schematic	17
9. Synthesis Snippets for each encoding.....	18
9.1. Gray Encoding	18
9.1.1. Synthesis Schematic.....	18
9.1.2. Timing Report summary	19
9.1.3. The critical path highlighted in the schematic	19
9.2. One_hot Encoding	20
9.2.1. Synthesis Schematic.....	20
9.2.2. Timing Report summary	21
9.2.3. The critical path highlighted in the schematic	21
9.3. Sequential Encoding	22
9.3.1. Synthesis Schematic.....	22
9.3.2. Timing Report summary	23
9.3.3. The critical path highlighted in the schematic	23
10. Implementation snippets for each encoding	24
10.1. Gray Encoding	24
10.1.1. Utilization report.....	24

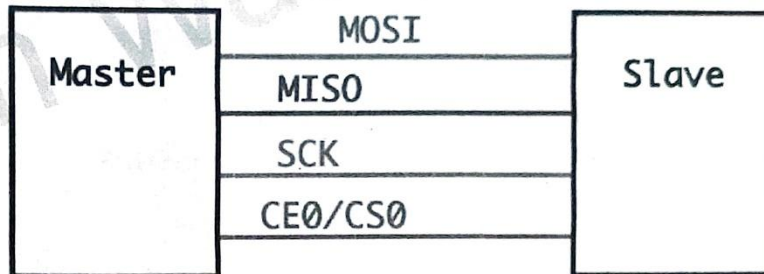
10.1.2.	Timing report snippet.....	24
10.1.3.	FPGA device snippet	24
10.2.	One_hot Encoding	25
10.2.1.	Utilization report.....	25
10.2.2.	Timing Report summary	25
10.2.3.	FPGA device snippet	25
10.3.	Sequential Encoding	26
10.3.1.	Utilization report.....	26
10.3.2.	Timing Report summary	26
10.3.3.	FPGA device snippet	26
11.	Snippet of the “Messages” tab	27
11.1.	Gray Encoding	27
11.2.	One_hot Encoding	27
11.3.	Sequential Encoding	27

1. Project Overview

1.1. Introduction

This project involves the design and implementation of a digital communication system using the Serial Peripheral Interface (SPI) protocol. The system consists of an SPI Slave module, a Single Port RAM, and an SPI Wrapper module that integrates these components. The implementation was carried out on an FPGA platform using Xilinx Vivado, demonstrating the design's functionality and performance.

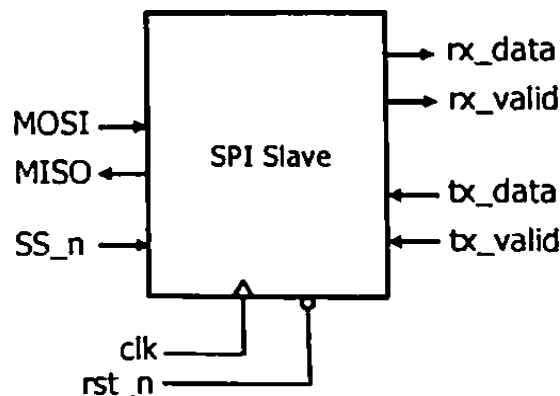
1.2. SPI Interface Diagram



- **Master:** The master device initiates the communication and controls the data transfer. It generates the clock signal (**SCK**) and selects the slave device by activating the chip select line (**CE0/CS0**). The master sends data to the slave through the **MOSI** (Master Out Slave In) line and receives data from the slave through the **MISO** (Master in Slave Out) line.
- **Slave:** The slave device responds to the master's commands. It receives data from the master via the **MOSI** line and sends data back to the master through the **MISO** line. The slave device is selected for communication when the **CE0/CS0** line is active (usually low).

1.3. Project Specifications

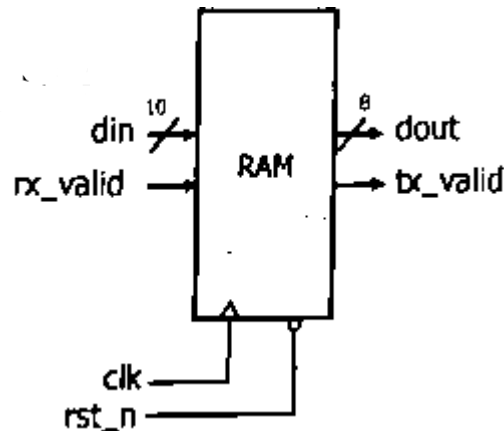
1.3.1. SPI Slave Interface



It handles the SPI protocol, converting serial data to parallel for storage in the RAM and vice versa. This block interfaces with the master device.

Name	Type	Size	Description
MOSI	Input	1 bit	Master output slave input signal
SS_n		1 bit	Active low Slave select signal
clk		1 bit	Clock Signal
arst_n		1 bit	Active low asynchronous reset signal
tx_data		8 bits	Transmitted data required from the RAM to the Master
tx_valid		1 bit	HIGH only when the data is ready to be received from RAM
rx_data	Output	10 bits	Received data from the Master converted from serial into parallel to the RAM
rx_valid		1 bit	HIGH only when the data is ready to be sent to RAM
MISO		1 bit	Master input slave output signal

1.3.2. Single Port Ram



It acts as the storage medium where the SPI Slave stores the received data and retrieves it upon request.

- Memory Depth: 256 words.
- Address Size: 8 bits.
- Data Width: 8-bit data stored and retrieved.
- Control Signals: rx_valid for writing data, tx_valid for reading data.

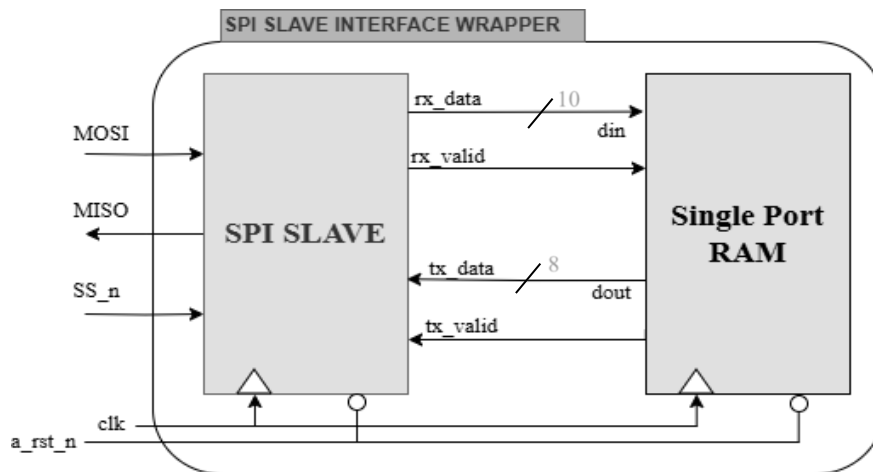
Parameters	
MEM_DEPTH	Default: 256
ADDR_SIZE	Default: 8

Name	Type	Size	Description
clk	Input	1 bit	Clock Signal
arst_n		1 bit	Active low asynchronous reset signal
din		10 bits	Data Input
rx_valid		1 bit	HIGH only accept din[7:0] to save the write/read address internally or write a memory word depending on the most significant 2 bits din[9:8]
dout	Output	8 bits	Data Output
tx_valid		1 bit	Whenever the command is memory read the tx_valid should be HIGH

- Din[9:8] selects the mode for Read/ Write on the single port asynchronous RAM

din[9:8]	Command	Description
00	Write	Hold din[7:0] internally as write address
01		Write din[7:0] in the memory with write address held previously
10	Read	Hold din[7:0] internally as read address
11		Read the memory with read address held previously, tx_valid should be HIGH, dout holds the word read from the memory, ignore din[7:0].

1.3.3. SPI Wrapper



It serves as the top-level module, managing the data flow between the SPI Slave and the RAM. It connects the SPI Slave's outputs (like rx_data and rx_valid) to the RAM's inputs (din, rx_valid), and the RAM's outputs (dout, tx_valid) to the SPI Slave's inputs (tx_data, tx_valid). It integrates the SPI Slave and RAM, with external connections for MOSI, MISO, SS_n, clk, and arst_n.

1.4. Design Flow

1. **Design Entry:** Verilog HDL was used to code the SPI Slave, Single Port RAM, and SPI Wrapper modules.
2. **Synthesis:** The design was synthesized in Xilinx Vivado, generating the netlist for FPGA implementation.
3. **Implementation:** Placement and routing were performed in Vivado, with timing constraints applied and verified.
4. **Simulation:** Functional simulations were carried out using testbenches, focusing on verifying SPI operations and memory interactions.
5. **Testing:** The system was tested on FPGA, validating the data integrity and timing requirements.

1.5. Summary

This project demonstrates the successful implementation of an SPI communication system on an FPGA. The SPI Slave module effectively interfaces with the master device, while the Single Port RAM stores and retrieves data as required. The SPI Wrapper integrates these components, ensuring smooth data transfer. The project was fully synthesized, implemented, and tested using Xilinx Vivado, with results meeting the design specifications.

3. RTL Design

3.1. Single Port Ram module

```
1 module single_port_Ram #(
2     /*-----Parameters-----*/
3     parameter MEM_DEPTH = 256,          /* Memory depth */
4     parameter ADD_SIZE = 8              /* Address size based upon the memory depth */
5 )
6 /*-----Inputs-----*/
7 input  [9:0]din,          /* Data input, din[9] determines write or read, 0 => Write, 1 => read
8                          * din[9:8] = 00 => Write, Hold din[7:0] internally as write address
9                          * din[9:8] = 01 => Write, write din[7:0] in the memory with wr address held previously
10                         * din[9:8] = 10 => Read, Hold din[7:0] internally as Read address
11                         * din[9:8] = 01 => Read the memory with rd address held previously,tx_valid = HIGH,
12                         *                               dout holds the word read from the memory, ignore din[7:0] */
13 input  clk,              /* clock signal input */
14 input  arst_n,           /* active low asynchronous reset */
15 input  rx_valid,         /* if HIGH: accept din[7:0] to save the wr/rd address internally or write a memory word */
16 /*-----outputs-----*/
17 output reg [7:0]dout,     /* data out of Ram */
18 output reg tx_valid       /* Whenever the command is memory read, the tx_valid should be HIGH */
19 );
20 /* internal bus to hold the address internally */
21 reg [ADD_SIZE-1:0]addr_internal;
22
23 /* memory declaration */
24 (* ram_style = "block" *)reg [7:0]mem[MEM_DEPTH-1:0];
25
26
27 always @(posedge clk) begin
28     if(~arst_n)begin
29         dout <= 0;
30         tx_valid <= 0;
31     end else if(rx_valid) begin
32         case (din[9:8])
33             2'b00 :
34                 /* Write operation - hold the write address */
35                 addr_internal <= din[7:0];
36             2'b01 :
37                 /* Write operation - write data to memory in the internal address held previously */
38                 mem[addr_internal] <= din[7:0];
39             2'b10 :
40                 /* Read operation - hold the read address */
41                 addr_internal <= din[7:0];
42             2'b11 : begin
43                 /* Read operation - read data from memory mem[addr_internal] */
44                 dout <= mem[addr_internal];
45                 tx_valid <= 1;
46             end
47             default: begin
48                 /* default case */
49                 dout <= 0;
50                 tx_valid <= 0;
51             end
52         endcase
53     end else begin
54         /* reset tx_valid when the rx_valid is low */
55         tx_valid <= 0;
56     end
57 end
58
59 endmodule
```


3.2. SPI Slave Interface “Sequential Encoding”

Sequential encoding has the highest setup slack time after implementation.

```
1 module SPI_Slave (
2     /*-----Inputs-----*/
3     input MOSI,      /* the serial date sent from the master */
4     input SS_n,      /* start and end communication from master side */
5     input [7:0]tx_data, /* the data to write in the memory */
6     input tx_valid,   /* the signal dedicate that tx_data is ready to covert from parallel to serial by slave*/
7     input clk,        /* clock signal input */
8     input arst_n,     /* active low synchronous reset */
9     /*-----outputs-----*/
10    output reg MISO,   /* the serial data sent to the master */
11    output reg [9:0]rx_data, /* the data which is read from the memory */
12    output reg rx_valid /* the signal dedicates that rx_data coverted to parallel by slave and ready for memory */
13 );
14 /*-----FSM States Declaration-----*/
15 localparam IDLE = 3'b000;
16 localparam CHX_CMD = 3'b001;
17 localparam WRITE = 3'b010;
18 localparam READ_ADD = 3'b011;
19 localparam READ_DATA = 3'b100;
20
21 /* Xilinx Vivado's Attribute FSM Encoding Method (Gray, One_Hot, Sequential) */
22 (* fsm_encoding = "sequential" *)
23 /*-----internal signals-----*/
24 reg [2:0]CS_NS; /* Current and Next States */
25 reg [3:0]rx_counter; /* to access the rx_data bus (8-bit) during converting from serial to parallel */
26 reg [3:0]tx_counter; /* to access the tx_data bus (8-bit) during converting from serial to parallel */
27 reg rd_addr_hold; /* Hold read address */
28 /*-----State memory-----*/
29 always @(posedge clk ) begin
30     if(~arst_n)begin
31         CS <= IDLE;
32     end else
33         CS <= NS;
34 end
35 /*-----Next State Logic-----*/
36 always @(*) begin
37     case (CS)
38         IDLE :begin
39             if(SS_n)
40                 NS = IDLE;
41             else
42                 NS = CHX_CMD;
43         end
44         CHX_CMD : begin
45             /* the master opens the communication to the slave */
46             if(~SS_n) begin
47                 /* if MOSI is low, operation will be write */
48                 if(~MOSI)
49                     NS = WRITE;
50                 /* if MOSI is high, operation will be read */
51             else begin
52                 /* if the read addr is held, the next is read the data */
53                 if(rd_addr_hold)
54                     NS = READ_DATA;
55                 /* if the read addr isn't held, the next is read address */
56                 else
57                     NS = READ_ADD;
58             end
59         end
60         /* the master ends the communication to the slave */
61         else
62             NS = IDLE;
63     end
64     WRITE : begin
65         /* the master opens the communication to the slave */
66         if(~SS_n) begin
67             NS = WRITE;
68         end
69         /* the master ends the communication to the slave */
70         else
71             NS = IDLE;
72     end
73 end
```

```

73     READ_ADD : begin
74         /* the master opens the communication to the slave */
75         if(~SS_n) begin
76             NS = READ_ADD;
77         end
78         /* the master ends the communication to the slave */
79         else
80             NS = IDLE;
81         end
82     READ_DATA : begin
83         /* the master opens the communication to the slave */
84         if(~SS_n)
85             NS = READ_DATA;
86         /* the master ends the communication to the slave */
87         else
88             NS = IDLE;
89         end
90     default: NS = IDLE;
91 endcase
92 end
93 /*-----Output logic-----*/
94 always @(posedge clk ) begin
95     if(~arst_n)begin
96         MISO          <= 0;
97         rx_data        <= 0;
98         rx_valid       <= 0;
99         rd_addr_hold   <= 0;
100        rx_counter     <= 0;
101        tx_counter     <= 0;
102    end
103    case (CS)
104        IDLE :begin
105            MISO      <= 0;
106            rx_data   <= 0;
107            rx_valid  <= 0;
108            rx_counter <= 0;
109            tx_counter <= 0;
110        end
111        CHX_CMD:begin
112            MISO      <= 0;
113            rx_data   <= 0;
114            rx_valid  <= 0;
115            rx_counter <= 0;
116            tx_counter <= 0;
117        end
118        WRITE:begin
119            if(rx_counter<10)begin
120                rx_data[9-rx_counter] <= MOSI ;
121                rx_counter <= rx_counter + 1;
122                rx_valid <= 0;
123            end else begin
124                rx_valid <= 1;
125                rx_counter <= 0;
126            end
127        end
128        READ_ADD:begin
129            if(rx_counter<10)begin
130                rx_data[9-rx_counter] <= MOSI ;
131                rx_counter <= rx_counter + 1;
132            end else begin
133                rx_valid <= 1;
134                rd_addr_hold <=1;
135                rx_counter <= 0;
136            end
137        end
138        READ_DATA:begin
139            if(rx_counter<10)begin
140                rx_data[9-rx_counter] <= MOSI ;
141                rx_counter <= rx_counter + 1;
142            end else begin
143                rx_valid <= 1;
144                /* Convert the read data from parallel to serial */
145                if(tx_valid)begin
146                    if(tx_counter<10)begin
147                        MISO <= tx_data[tx_counter] ;
148                        tx_counter <= tx_counter + 1;
149                    end else begin
150                        MISO <= 0;
151                        rx_counter <= 0;
152                        tx_counter <= 0;
153                        rd_addr_hold <= 0;
154                        rx_valid <= 0;
155                    end
156                end
157            end
158        end

```

```

159     default:begin
160         MISO <= 0;
161         rx_data <= 0;
162         rx_valid <= 0;
163         tx_counter <=0;
164         rx_counter <=0;
165     end
166 endcase
167 end
168
169 endmodule //SPI_Slave

```

3.3. SPI Wrapper module

```

1  module SPI_Wrapper #(
2      /*-----Parameters-----*/
3      parameter MEM_DEPTH = 256,      /* Memory depth */
4      parameter ADD_SIZE = 8          /* Address size based upon the memory depth */
5  )(
6      /*-----Inputs-----*/
7      input MOSI,          /* the serial date sent from the master */
8      input SS_n,          /* start and end communication from master side */
9      input clk,           /* clock signal input */
10     input arst_n,        /* active low asynchronous reset */
11     /*-----Outputs-----*/
12     output MISO           /* the serial data sent to the master */
13 );
14 /*-----internal signals which connect Ram with SPI Slave-----*/
15 wire [7:0]tx_data;      /* connect output of ram "dout" with input of Slave "tx_data" */
16 wire tx_valid;          /* connect output of ram "tx_valid" with input of Slave "tx_valid" */
17 wire [9:0]rx_data;      /* connect input of ram "din" with output of Slave "rx_data" */
18 wire rx_valid;          /* connect input of ram "rx_valid" with output of Slave "rx_valid" */
19
20 /*-----SPI_Slave Instantiation-----*/
21 SPI_Slave SPI_slave (
22     .MOSI(MOSI),          /* the serial date sent from the master */
23     .SS_n(SS_n),          /* start and end communication from master side */
24     .tx_data(tx_data),    /* the data to write in the memory */
25     .tx_valid(tx_valid),  /* the signal dedicate that tx_data is ready to covert from parallel to serial by slave*/
26     .clk(clk),            /* clock signal input */
27     .arst_n(arst_n),      /* active low synchronous reset */
28     .MISO(MISO),          /* the serial data sent to the master */
29     .rx_data(rx_data),    /* the data which is read from the memory */
30     .rx_valid(rx_valid)   /* the signal dedicates that rx_data covert to parallel by slave and ready for memory */
31 );
32
33 /*-----Ram Instantiation-----*/
34 single_port_Ram #(
35     .MEM_DEPTH(MEM_DEPTH),
36     .ADD_SIZE(ADD_SIZE))
37 RAM (
38     .din(rx_data),        /* Ram Data input */
39     .clk(clk),            /* clock signal input */
40     .arst_n(arst_n),      /* active low asynchronous reset */
41     .rx_valid(rx_valid),  /* informs ram that data input is valid */
42     .dout(tx_data),       /* Ram Data output */
43     .tx_valid(tx_valid)   /* dedicated that data output is valid */
44 );
45
46 endmodule //SPI_Wrapper
47

```

4. SPI Wrapper Testbench as Master

```
1 module SPI_Master_tb ();
2 /*-----Parameters-----*/
3 parameter MEM_DEPTH = 256;
4 parameter ADD_SIZE = 8;
5 /*-----inputs-----*/
6 reg MOSI; /* the serial date sent from the master */
7 reg SS_n; /* start and end communication from master side */
8 reg clk; /* clock signal input */
9 reg arst_n; /* active low asynchronous reset */
10 /*-----outputs-----*/
11 wire MISO; /* the serial data sent to the master */
12 /*-----Internal signal-----*/
13 reg [9:0]data_addr_input; /* Data address input */
14 reg [7:0]Data_output; /* Data output bus after converting it parallel */
15 /*-----DUT INSTANTIATIONS-----*/
16 SPI_Wrapper #(
17 .MEM_DEPTH(MEM_DEPTH),
18 .ADD_SIZE(ADD_SIZE)
19 ) DUT (
20 .MOSI(MOSI),
21 .SS_n(SS_n),
22 .clk(clk),
23 .arst_n(arst_n),
24 .MISO(MISO)
25 );
26
27 /*-----CLOCK GENERATION-----*/
28 initial begin
29 clk = 0;
30 forever begin
31 #5; clk=~clk;
32 end
33 end
34
35 integer i; // use in the loop cnverting the parallel input to serial
36 /*-----TEST STIMULUS-----*/
37 initial begin
38 $display("----Start Simulation----\n");
39
40 /*==== Inputs Intialization =====*/
41 MOSI = 0;
42 SS_n = 0;
43 arst_n = 1;
44 data_addr_input = 0;
45 Data_output <= 0;
46
47 /*====check the reset signal====*/
48 $display("Check the reset signal");
49 arst_n = 0; // Activate reset
50 repeat(3) @(negedge clk);
51 self_checking(MISO,0);
52 arst_n = 1; // Diactivate reset
53
54 /*====check if the master doesn't communicate with slave====*/
55 $display("The master didn't begin communications with Slave");
56 SS_n = 1; // End Communication with Slave
57 repeat(2) @(negedge clk);
58 MOSI = 1;
59 @(negedge clk);
60 self_checking(MISO,0);
61
62 /*===== WRITE OPERATION TESTING =====*/
63 $display("The master begin communications with Slave");
64 SS_n = 0; // Start Communication with Slave
65 @(negedge clk);
66 MOSI = 0; // to inform the slave
67 @(negedge clk);
68
69 /* send the write address to slave */
70 $display("The master sends a write address");
71 data_addr_input = 10'b00_1010_1001; // data_addr_input[9:8] must be 00
72 /* For loop to convert the data_addr_input bus to serial data per clk */
73 for( i=0 ; i<10 ; i=i+1 )begin
74 MOSI = data_addr_input[9-i];
75 @(negedge clk);
76 end
77
78 MOSI = 1; // clear MOSI
79 @(negedge clk); // Hold SS_n low for one more clock cycle
80 SS_n = 1; // End Communication with Slave
81 repeat(3) @(negedge clk);
82
83 /* send the data to slave to write in address held previously */
84 $display("The master sends a data to store in the address sent previously");
85 SS_n = 0; // Start Communication with Slave
86 @(negedge clk);
87 MOSI = 0; // to dedict write operation
88 @(negedge clk);
89
90 data_addr_input = 10'b01_1111_0001; // data_addr_input[9:8] must be 01
91 /* For loop to convert the data_addr_input bus to serial data per clk */
92 for( i=0 ; i<10 ; i=i+1 )begin
93 MOSI = data_addr_input[9-i];
94 @(negedge clk);
95 end
96
```

```

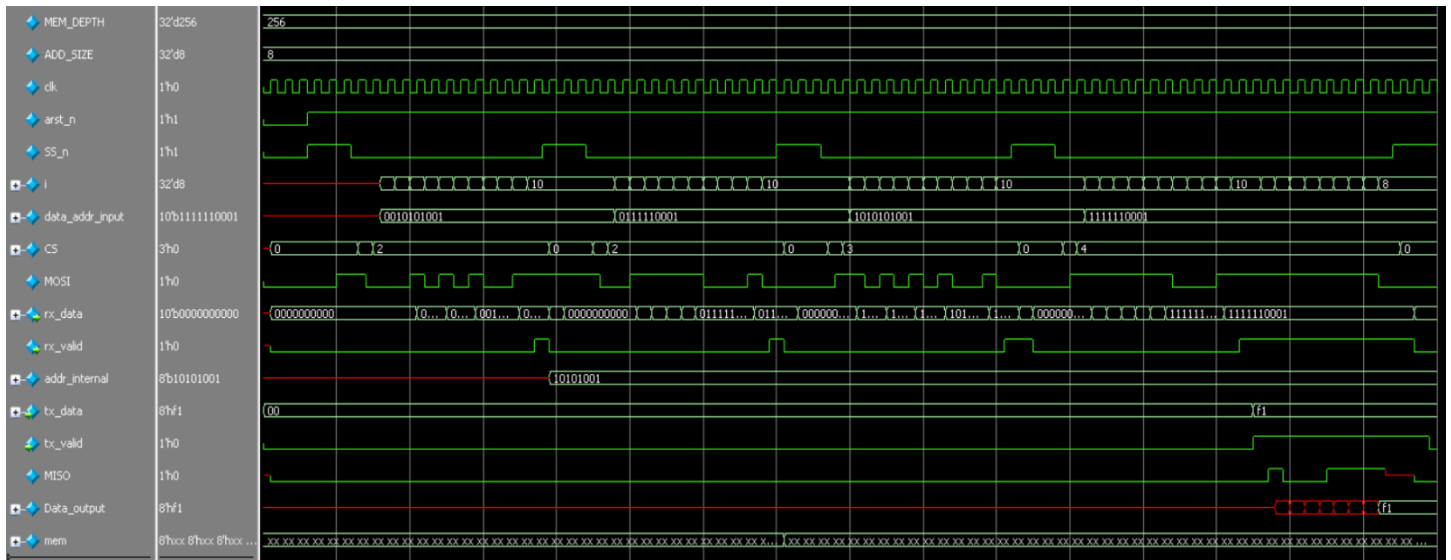
97     MOSI = 0;           // clear MOSI
98     @(negedge clk);     // Hold SS_n low for one more clock cycle
99     SS_n = 1;           // End Communication with Slave
100    repeat(3) @(negedge clk);
101
102    $display("\nCheck address '169' in the memory. It should have data '1111_0001'='F1' ");
103    self_checking_8bit(DUT.RAM.mem[169],8'hF1);
104
105    /*===== Read OPERATION TESTING =====*/
106    $display("The master begin communications with Slave");
107    SS_n = 0;           // Start Communication with Slave
108    @(negedge clk);
109    MOSI = 1;           // to inform the slave there is read operation
110    @(negedge clk);
111    /* send the write address to slave */
112    $display("The master sends a read address");
113    data_addr_input = 10'b10_1010_1001; // data_addr_input[9:8] must be 10
114    /* For loop to convert the data_addr_input bus to serial data per clk */
115    for( i=0 ; i<10 ; i=i+1 )begin
116        MOSI = data_addr_input[9-i];
117        @(negedge clk);
118    end
119
120    MOSI = 0;           // clear MOSI
121    @(negedge clk);     // Hold SS_n low for one more clock cycle
122    SS_n = 1;           // End Communication with Slave
123    repeat(3) @(negedge clk);
124
125    /* send the data bits code to slave to read from address held previously */
126    $display("The master sends a data code bits to read from the address sent previously");
127    SS_n = 0;           // Start Communication with Slave
128    @(negedge clk);
129    MOSI = 1;           // to dedict write operation
130    @(negedge clk);
131
132    data_addr_input = 10'b11_1111_0001; // data_addr_input[9:8] must be 11 and other bits are dummy
133    /* For loop to convert the data_addr_input bus to serial data per clk */
134    for( i=0 ; i<10 ; i=i+1 )begin
135        MOSI = data_addr_input[9-i];
136        @(negedge clk);
137    end
138
139    $display("\nCheck address '169' in the memory. and the MISO bits transmitted");
140
141    @(negedge clk);     // Ensure data is stable
142    @(negedge clk);     // to update tx_data
143
144    /* For loop to convert the MISO from serial data per clk to the internal signal 'Data_output' bus */
145    for(i=0; i<8; i=i+1) begin
146        @(negedge clk);
147        Data_output[i] = MISO;
148    end
149
150    MOSI = 0;           // clear MOSI
151    @(negedge clk);     // Hold SS_n low for one more clock cycle
152    SS_n = 1;           // End Communication with Slave
153    repeat(3) @(negedge clk);
154    self_checking_8bit(Data_output,DUT.RAM.mem[169]);
155
156    $display("----End Simulation----\n");
157    $stop;
158
159 end
160
161 /*-----Self_Checking 1-bit-----*/
162 task self_checking;
163     input DUT_out;
164     input expected_tb;
165     begin
166         // Check if the output is correct
167         if (DUT_out == expected_tb) begin
168             $display("\n---Output is correct---");
169             $display("SPI_out = %b, SPI_Expected = %b\n", DUT_out,expected_tb);
170         end
171         else begin
172             $display("Error!!!\n---Output is incorrect---");
173             $display("SPI_out = %b, SPI_Expected = %b\n", DUT_out,expected_tb);
174             $stop;
175         end
176     end
177 endtask
178
179 /*-----Self_Checking 8-bit-----*/
180 task self_checking_8bit;
181     input [7:0]DUT_out;
182     input [7:0]expected_tb;
183     begin
184         // Check if the output is correct
185         if (DUT_out == expected_tb) begin
186             $display("\n---Output is correct---");
187             $display("SPI_out = %x, SPI_Expected = %x", DUT_out,expected_tb);
188         end
189         else begin
190             $display("Error!!!\n---Output is incorrect---");
191             $display("SPI_out = %x, SPI_Expected = %x", DUT_out,expected_tb);
192             $stop;
193         end
194     end
195 endtask
196
197 endmodule //SPI_Master_tb

```

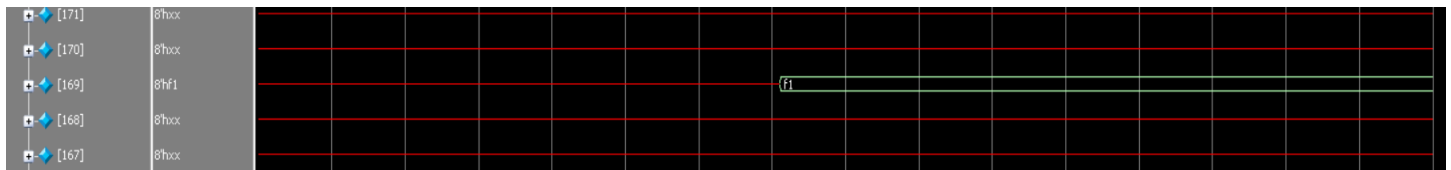
5. Do file to run testbench

```
1  #create Work Folder
2  vlib work
3
4  #Compile files with names
5  vlog Single_Port_Ram.v SPI_Slave.v SPI_Wrapper.v SPI_Master_tb.v
6
7  #simulate The TB file with module Name
8  vsim -voptargs=+acc work.SPI_Master_tb
9
10 #add the variables and internal signals with specific order to notice them easily
11
12 add wave -position insertpoint \
13 sim:/SPI_Master_tb/MEM_DEPTH \
14 sim:/SPI_Master_tb/ADD_SIZE \
15 sim:/SPI_Master_tb/clk \
16 sim:/SPI_Master_tb/arst_n \
17 sim:/SPI_Master_tb/SS_n \
18 sim:/SPI_Master_tb/i \
19 sim:/SPI_Master_tb/data_addr_input \
20 sim:/SPI_Master_tb/DUT/SPI_slave/CS \
21 sim:/SPI_Master_tb/MOSI \
22 sim:/SPI_Master_tb/DUT/SPI_slave/rx_data \
23 sim:/SPI_Master_tb/DUT/SPI_slave/rx_valid \
24 sim:/SPI_Master_tb/DUT/RAM/addr_internal \
25 sim:/SPI_Master_tb/DUT/SPI_slave/tx_data \
26 sim:/SPI_Master_tb/DUT/SPI_slave/tx_valid \
27 sim:/SPI_Master_tb/MISO \
28 sim:/SPI_Master_tb/Data_output \
29 sim:/SPI_Master_tb/DUT/RAM/mem \
30
31
32 run -all
33
34 wave zoom full
```

6. Snippets from the waveforms



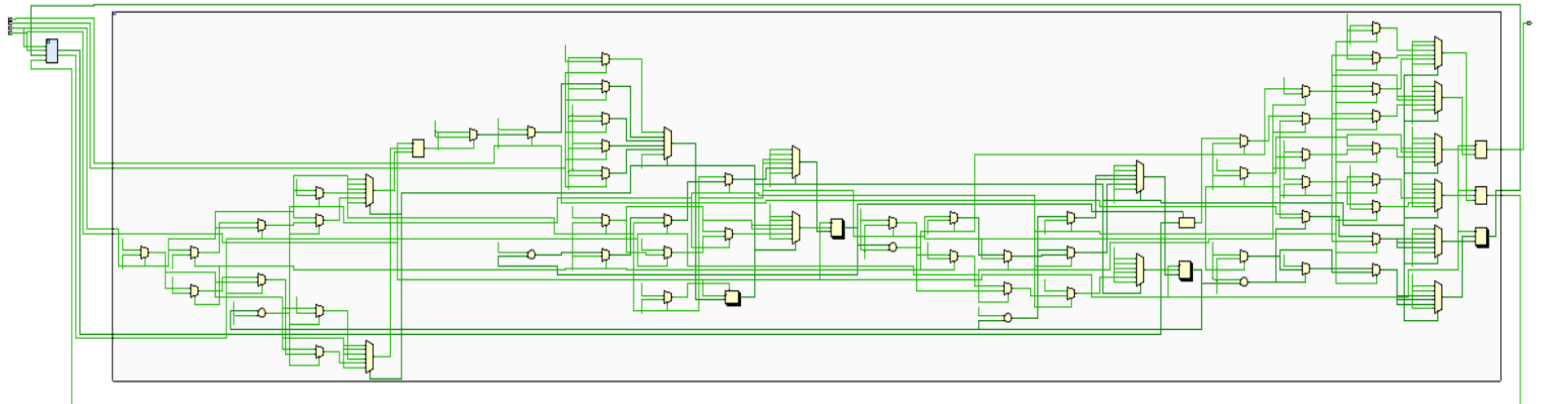
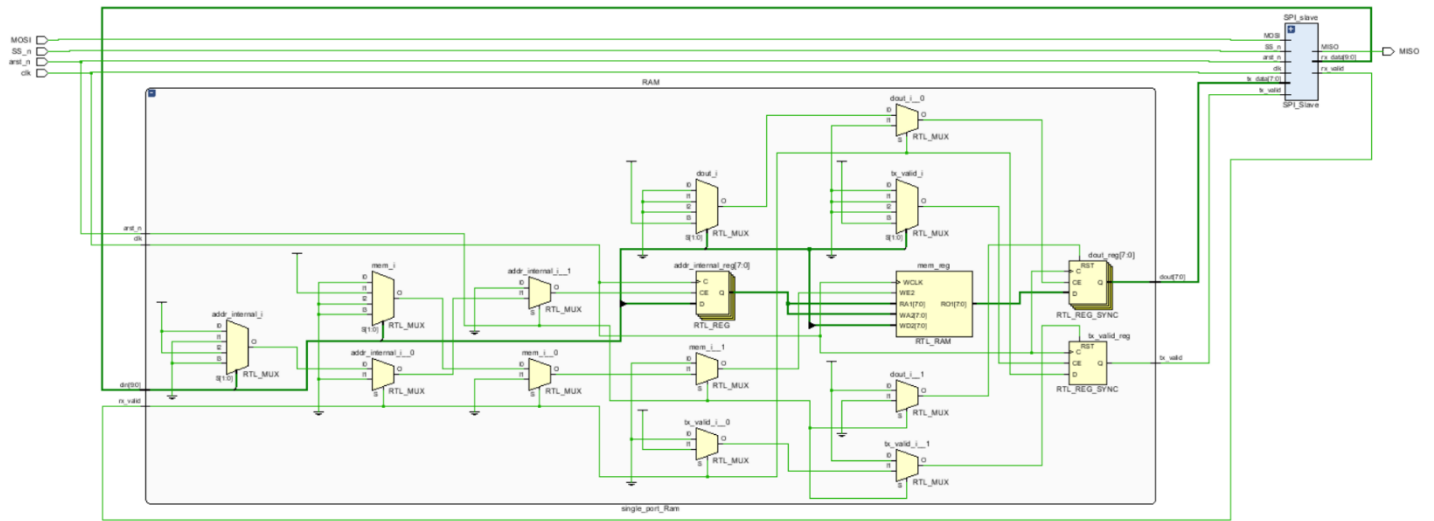
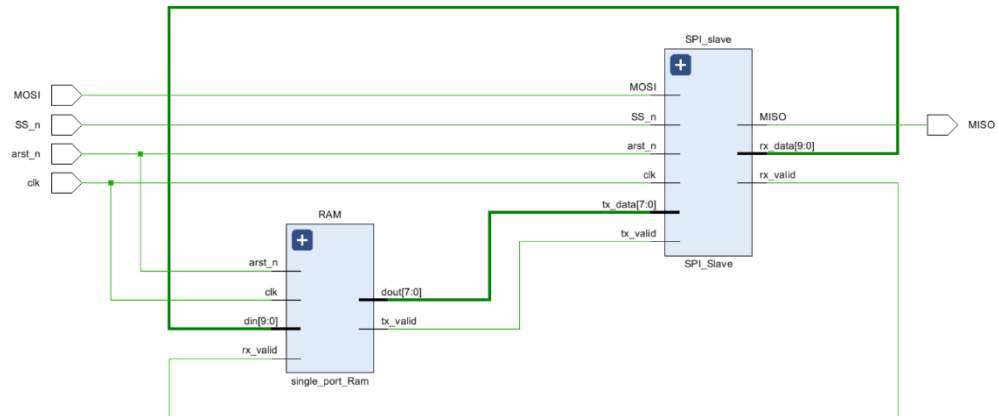
Showing the value stored in address “169”



7. Constrains File after adding a debug core “sequential encoding”

```
1  ## This file is a general .xdc for the Basys3 rev B board
2  ## To use it in a project:
3  ## - uncomment the lines corresponding to used pins
4  ## - rename the used ports (in each line, after get_ports) according to the top level signal names in the project
5
6  ## Clock signal
7  set_property -dict {PACKAGE_PIN W5 IOSTANDARD LVCMOS33} [get_ports clk]
8  create_clock -period 10.000 -name clk -waveform {0.000 5.000} -add [get_ports clk]
9
10 ## Switches
11 set_property -dict {PACKAGE_PIN V17 IOSTANDARD LVCMOS33} [get_ports arst_n]
12 set_property -dict {PACKAGE_PIN V16 IOSTANDARD LVCMOS33} [get_ports SS_n]
13 set_property -dict {PACKAGE_PIN W16 IOSTANDARD LVCMOS33} [get_ports MOSI]
14 #set_property -dict { PACKAGE_PIN W17 IOSTANDARD LVCMOS33 } [get_ports {sw[3]}]
15
16 ## LEDs
17 set_property -dict {PACKAGE_PIN U16 IOSTANDARD LVCMOS33} [get_ports MISO]
18 #set_property -dict { PACKAGE_PIN E19 IOSTANDARD LVCMOS33 } [get_ports {led[1]}]
19 #set_property -dict { PACKAGE_PIN U19 IOSTANDARD LVCMOS33 } [get_ports {led[2]}]
20 #set_property -dict { PACKAGE_PIN V19 IOSTANDARD LVCMOS33 } [get_ports {led[3]}]
21
22
23 ##Buttons
24 #set_property -dict { PACKAGE_PIN U18 IOSTANDARD LVCMOS33 } [get_ports rst]
25
26 ## Configuration options, can be used for all designs
27 set_property CONFIG_VOLTAGE 3.3 [current_design]
28 set_property CFBVS VCC0 [current_design]
29
30 ## SPI configuration mode options for QSPI boot, can be used for all designs
31 set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]
32 set_property BITSTREAM.CONFIG.CONFIGRATE 33 [current_design]
33 set_property CONFIG_MODE SPIx4 [current_design]
34
35 set_property MARK_DEBUG true [get_nets arst_n_IBUF]
36 set_property MARK_DEBUG true [get_nets clk_IBUF]
37 set_property MARK_DEBUG true [get_nets MISO_OBUF]
38 set_property MARK_DEBUG true [get_nets MOSI_IBUF]
39 set_property MARK_DEBUG true [get_nets SS_n_IBUF]
40 create_debug_core u_ila_0 ila
41 set_property ALL_PROBE_SAME_MU true [get_debug_cores u_ila_0]
42 set_property ALL_PROBE_SAME_MU_CNT 1 [get_debug_cores u_ila_0]
43 set_property C_ADV_TRIGGER false [get_debug_cores u_ila_0]
44 set_property C_DATA_DEPTH 1024 [get_debug_cores u_ila_0]
45 set_property C_EN_STRG_QUAL false [get_debug_cores u_ila_0]
46 set_property C_INPUT_PIPE_STAGES 0 [get_debug_cores u_ila_0]
47 set_property C_TRIGIN_EN false [get_debug_cores u_ila_0]
48 set_property C_TRIGOUT_EN false [get_debug_cores u_ila_0]
49 set_property port_width 1 [get_debug_ports u_ila_0/clk]
50 connect_debug_port u_ila_0/clk [get_nets [list clk_IBUF_BUF]]
51 set_property PROBE_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe0]
52 set_property port_width 1 [get_debug_ports u_ila_0/probe0]
53 connect_debug_port u_ila_0/probe0 [get_nets [list arst_n_IBUF]]
54 create_debug_port u_ila_0 probe
55 set_property PROBE_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe1]
56 set_property port_width 1 [get_debug_ports u_ila_0/probe1]
57 connect_debug_port u_ila_0/probe1 [get_nets [list clk_IBUF]]
58 create_debug_port u_ila_0 probe
59 set_property PROBE_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe2]
60 set_property port_width 1 [get_debug_ports u_ila_0/probe2]
61 connect_debug_port u_ila_0/probe2 [get_nets [list MISO_OBUF]]
62 create_debug_port u_ila_0 probe
63 set_property PROBE_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe3]
64 set_property port_width 1 [get_debug_ports u_ila_0/probe3]
65 connect_debug_port u_ila_0/probe3 [get_nets [list MOSI_IBUF]]
66 create_debug_port u_ila_0 probe
67 set_property PROBE_TYPE DATA_AND_TRIGGER [get_debug_ports u_ila_0/probe4]
68 set_property port_width 1 [get_debug_ports u_ila_0/probe4]
69 connect_debug_port u_ila_0/probe4 [get_nets [list SS_n_IBUF]]
70 set_property C_CLK_INPUT_FREQ_HZ 300000000 [get_debug_cores dbg_hub]
71 set_property C_ENABLE_CLK_DIVIDER false [get_debug_cores dbg_hub]
72 set_property C_USER_SCAN_CHAIN 1 [get_debug_cores dbg_hub]
73 connect_debug_port dbg_hub/clk [get_nets clk_IBUF_BUF]
74
```


8. Elaboration Schematic



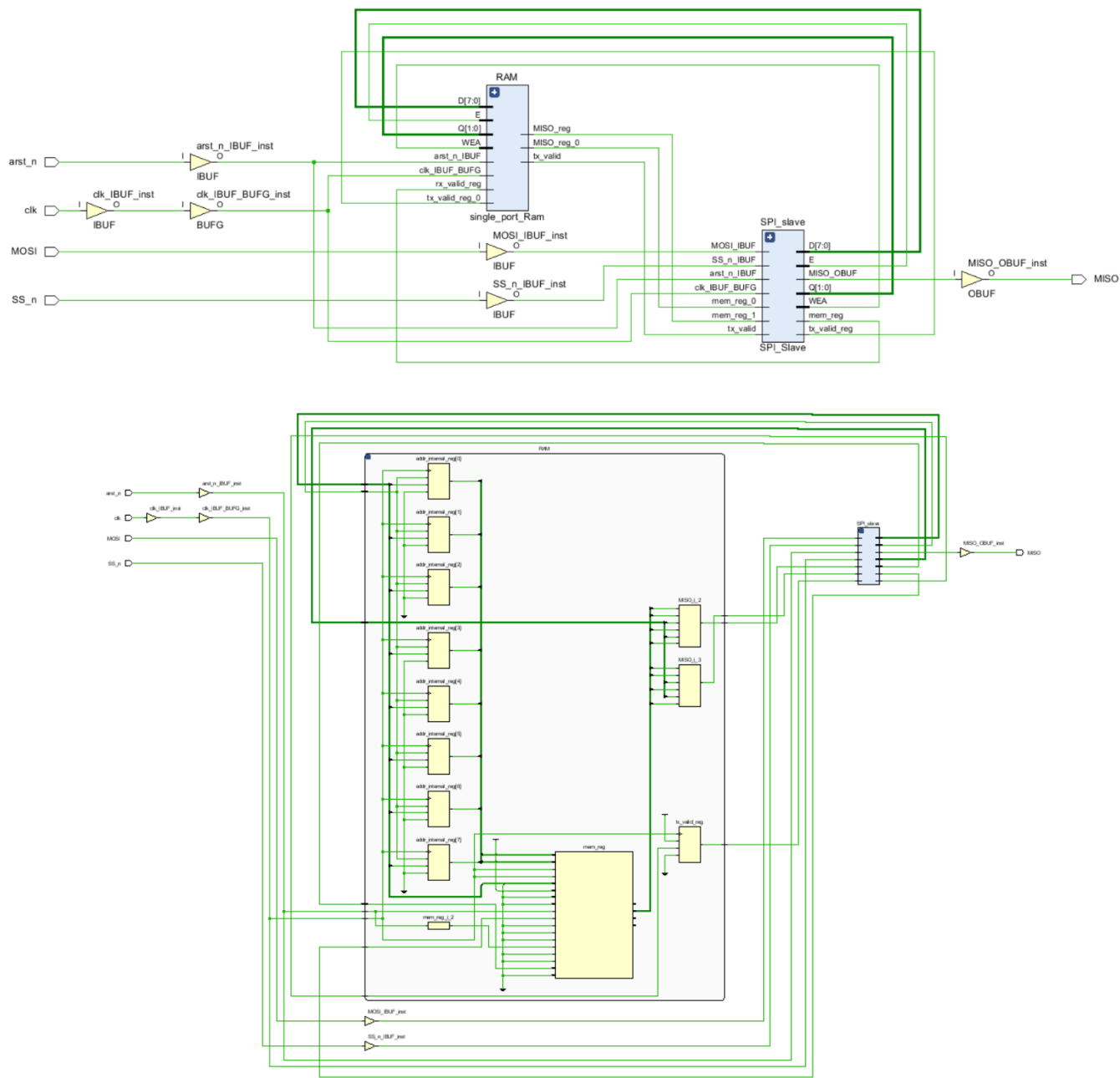
9. Synthesis Snippets for each encoding

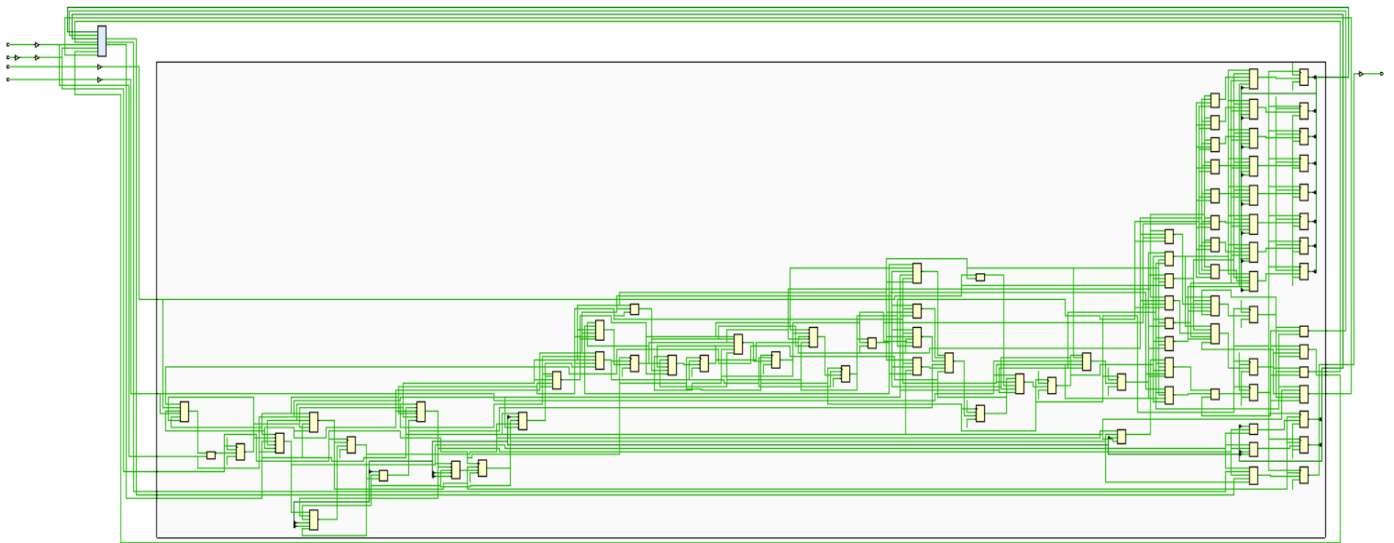
9.1. Gray Encoding

State	New Encoding	Previous Encoding
IDLE	000	000
CHX_CMD	001	001
WRITE	011	010
READ_DATA	010	100
READ_ADD	111	011

INFO: [Synth 8-3354] encoded FSM with state register 'CS_reg' using encoding 'gray' in module 'SPI_Slave'

9.1.1. Synthesis Schematic



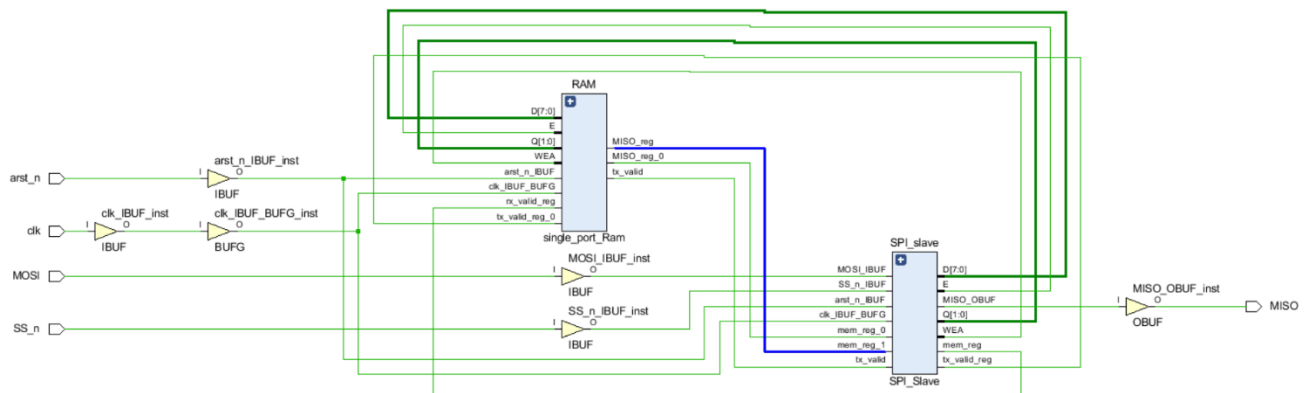


9.1.2. Timing Report summary

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 5.445 ns	Worst Hold Slack (WHS): 0.146 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 77	Total Number of Endpoints: 77	Total Number of Endpoints: 36
All user specified timing constraints are met.		

9.1.3. The critical path highlighted in the schematic

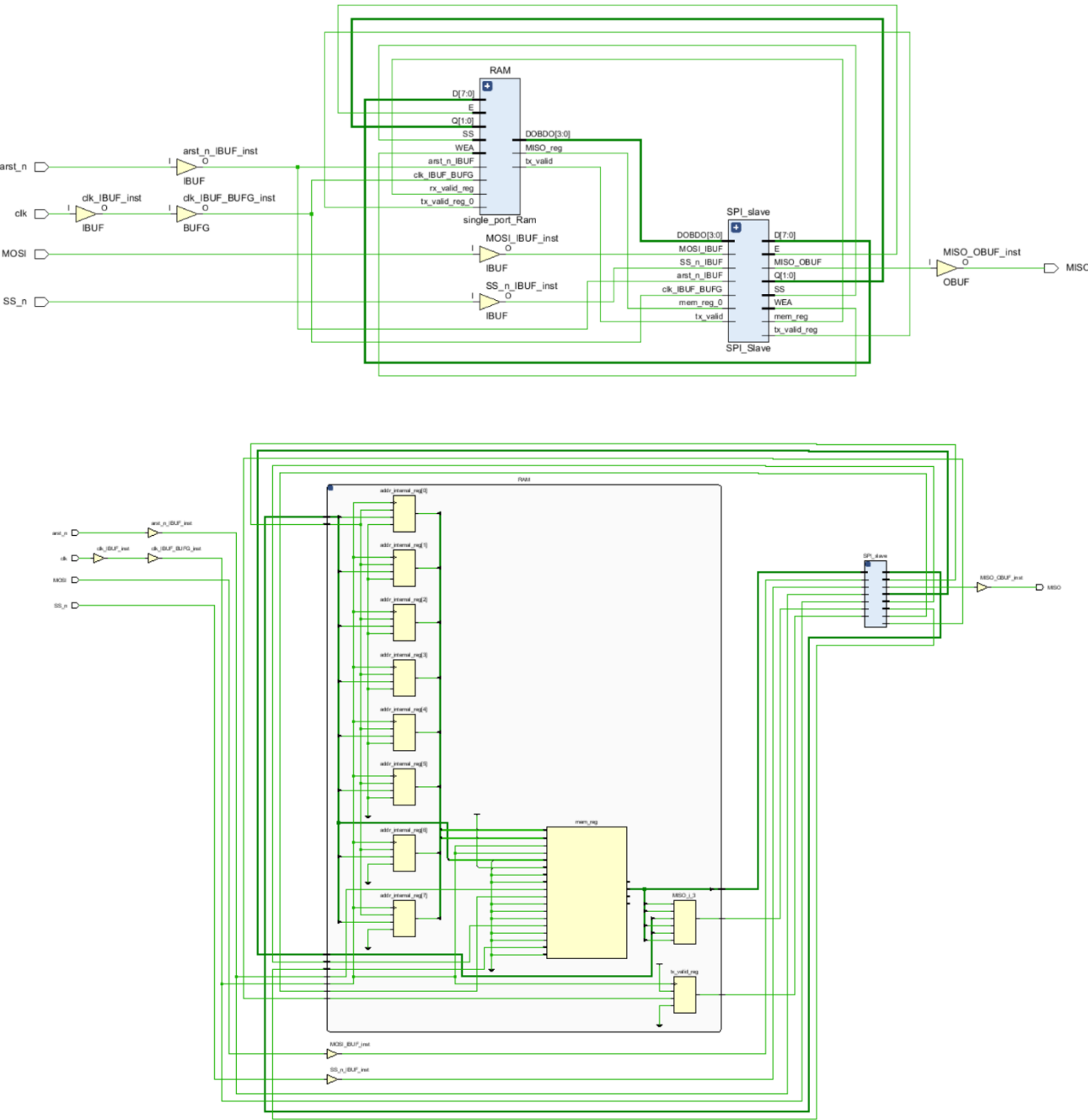


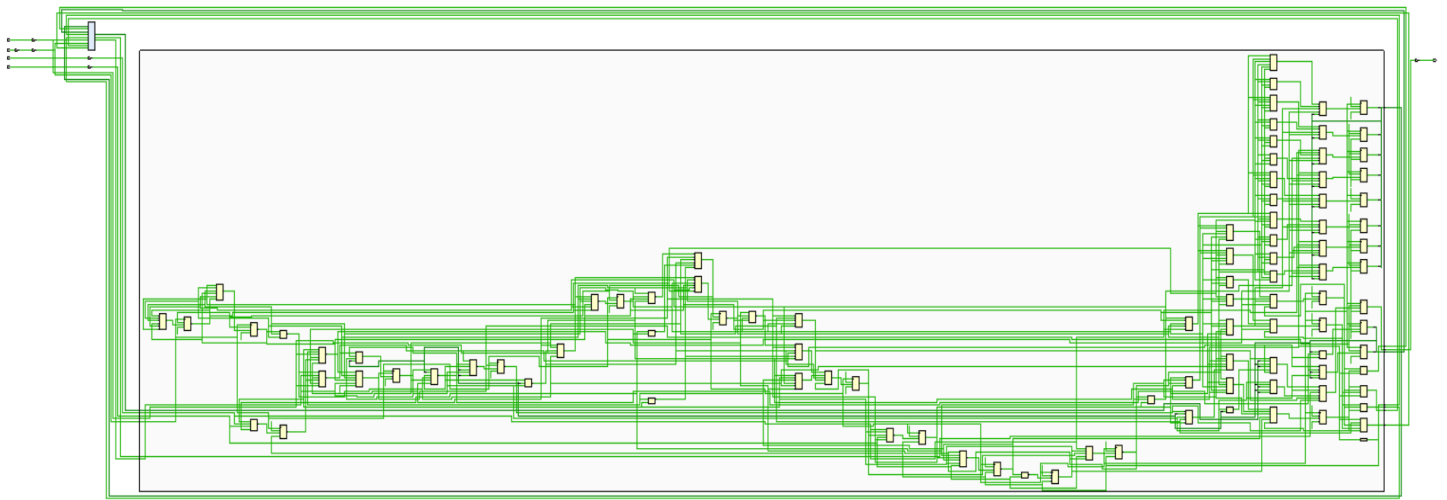
9.2. One_hot Encoding

State	New Encoding	Previous Encoding
IDLE	00001	000
CHX_CMD	00010	001
WRITE	00100	010
READ_DATA	01000	100
READ_ADD	10000	011

INFO: [Synth 8-3354] encoded FSM with state register 'CS_reg' using encoding 'one-hot' in module 'SPI_Slave'

9.2.1. Synthesis Schematic





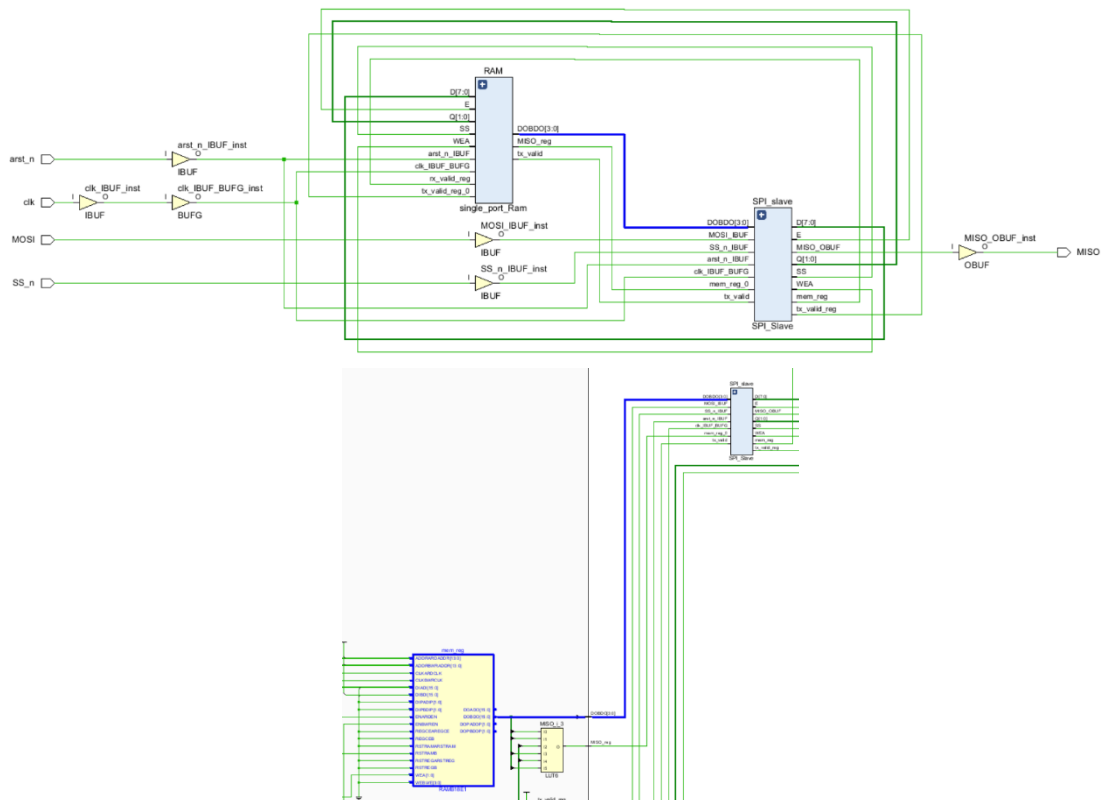
9.2.2. Timing Report summary

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 5.419 ns	Worst Hold Slack (WHS): 0.146 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 79	Total Number of Endpoints: 79	Total Number of Endpoints: 38

All user specified timing constraints are met.

9.2.3. The critical path highlighted in the schematic

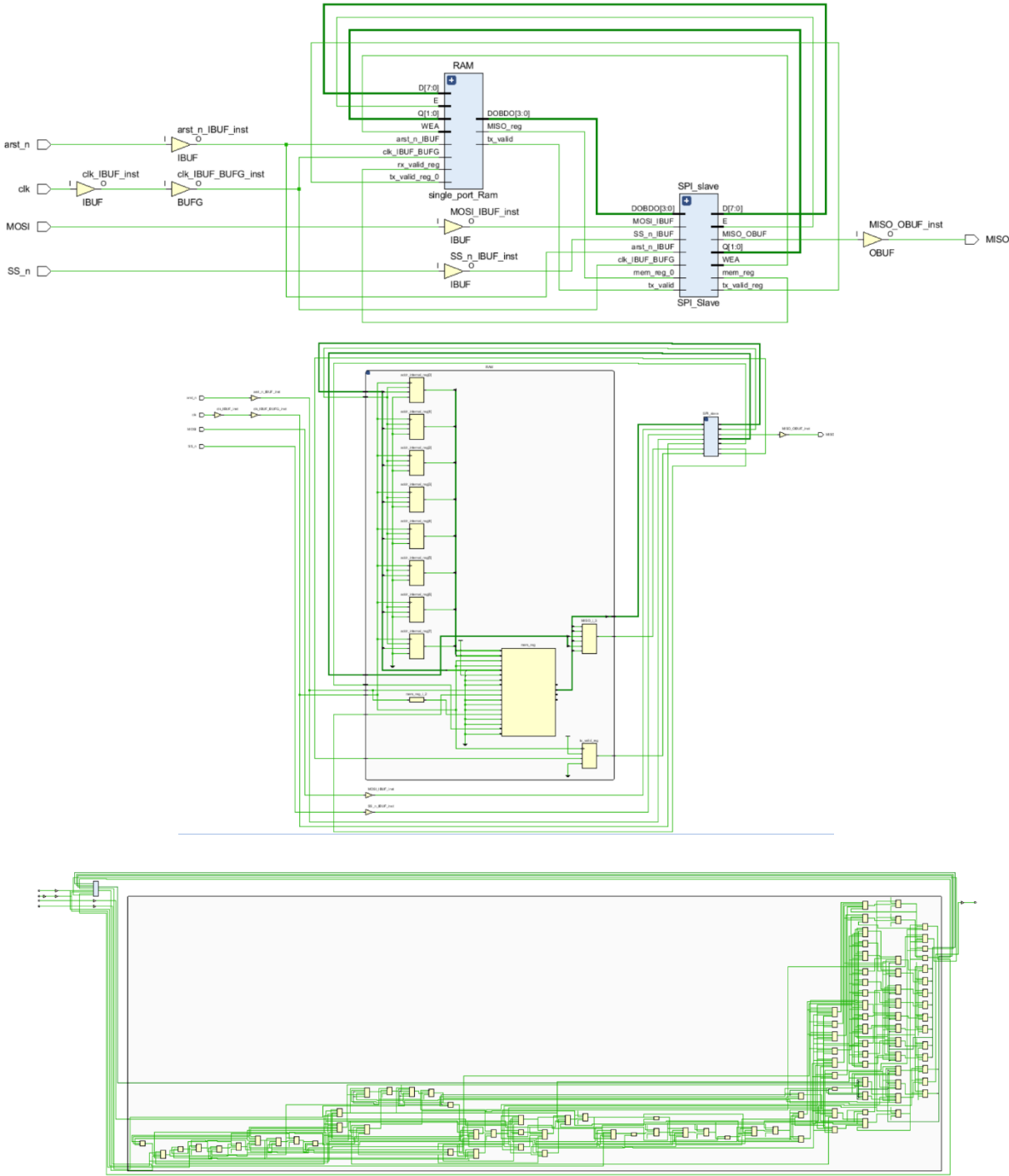


9.3. Sequential Encoding

State	New Encoding	Previous Encoding
IDLE	000	000
CHX_CMD	001	001
WRITE	010	010
READ_DATA	011	100
READ_ADD	100	011

: [Synth 8-3354] encoded FSM with state register 'CS_reg' using encoding 'sequential' in module 'SPI_Slave'

9.3.1. Synthesis Schematic



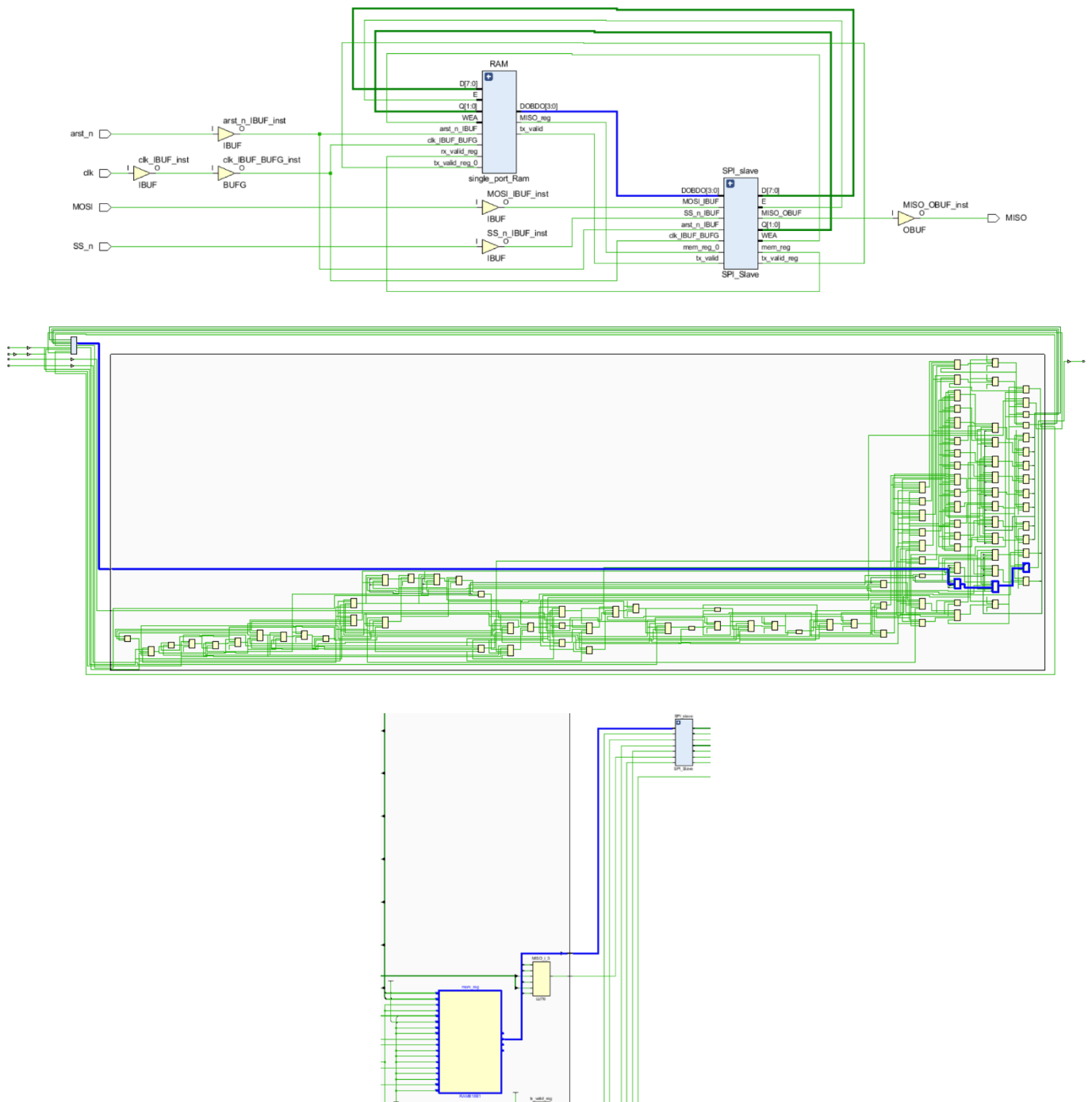
9.3.2. Timing Report summary

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 5.419 ns	Worst Hold Slack (WHS): 0.146 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 77	Total Number of Endpoints: 77	Total Number of Endpoints: 36

All user specified timing constraints are met.

9.3.3. The critical path highlighted in the schematic



10. Implementation snippets for each encoding

10.1. Gray Encoding

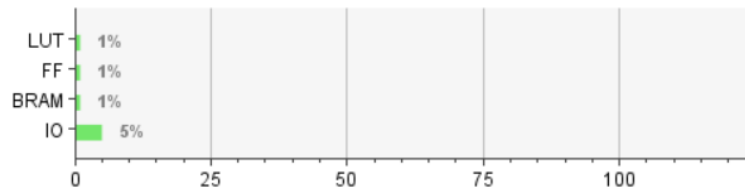
10.1.1. Utilization report

Q H % Hierarchy

Name	Slice LUTs (20800)	Slice Registers (41600)	Slice (8150)	LUT as Logic (20800)	LUT Flip Flop Pairs (20800)	Block RAM Tile (50)	Bonded IOB (106)	BUFGCTRL (32)
▼ N SPI_Wrapper	55	33	19	55	24	0.5	5	1
RAM (single_port_Ram)	3	9	5	3	0	0.5	0	0
SPI_slave (SPI_Slave)	52	24	17	52	23	0	0	0

Summary

Resource	Utilization	Available	Utilization %
LUT	55	20800	0.26
FF	33	41600	0.08
BRAM	0.50	50	1.00
IO	5	106	4.72



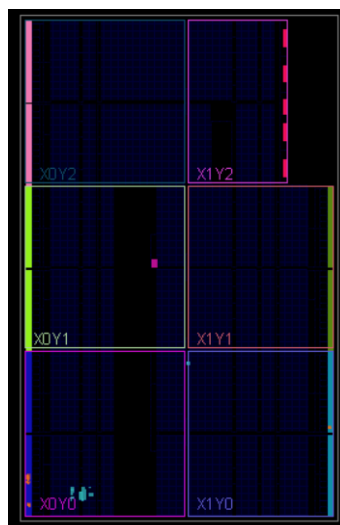
10.1.2. Timing report snippet

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 5.201 ns	Worst Hold Slack (WHS): 0.058 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 78	Total Number of Endpoints: 78	Total Number of Endpoints: 36

All user specified timing constraints are met.

10.1.3. FPGA device snippet



10.2. One_hot Encoding

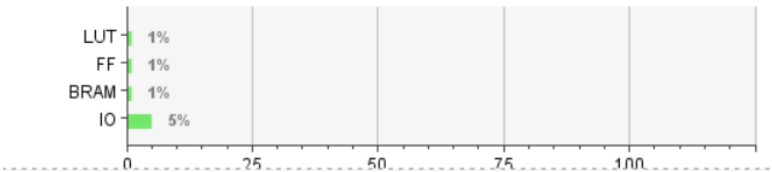
10.2.1. Utilization report

Hierarchy

Name	Slice LUTs (20800)	Slice Registers (41600)	Slice (8150)	LUT as Logic (20800)	LUT Flip Flop Pairs (20800)	Block RAM Tile (50)	Bonded IOB (106)	BUFCTRL (32)
▼ SPI_Wrapper	64	35	19	64	26	0.5	5	1
RAM (single_port_Ram)	2	9	3	2	0	0.5	0	0
SPI_slave (SPI_Slave)	62	26	18	62	25	0	0	0

Summary

Resource	Utilization	Available	Utilization %
LUT	64	20800	0.31
FF	35	41600	0.08
BRAM	0.50	50	1.00
IO	5	106	4.72



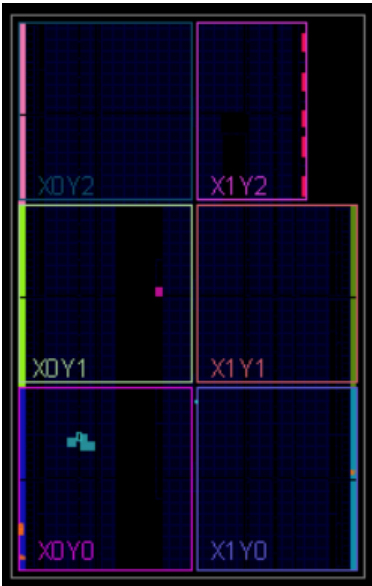
10.2.2. Timing Report summary

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 5.281 ns	Worst Hold Slack (WHS): 0.110 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 80	Total Number of Endpoints: 80	Total Number of Endpoints: 38

All user specified timing constraints are met.

10.2.3. FPGA device snippet



10.3. Sequential Encoding

10.3.1. Utilization report

Q

≡

≡

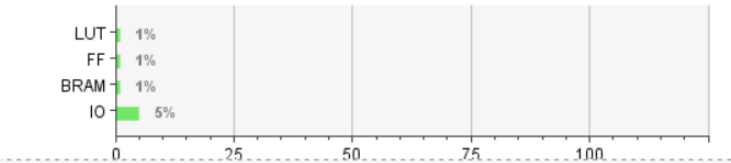
%

Hierarchy

Name	1	Slice LUTs (20800)	Slice Registers (41600)	Slice (8150)	LUT as Logic (20800)	LUT Flip Flop Pairs (20800)	Block RAM Tile (50)	Bonded IOB (106)	BUFGCTRL (32)
▼ N SPI_Wrapper		64	33	20	64	24	0.5	5	1
RAM (single_port_Ram)		2	9	3	2	0	0.5	0	0
SPI_slave (SPI_Slave)		62	24	20	62	23	0	0	0

Summary

Resource	Utilization	Available	Utilization %
LUT	64	20800	0.31
FF	33	41600	0.08
BRAM	0.50	50	1.00
IO	5	106	4.72



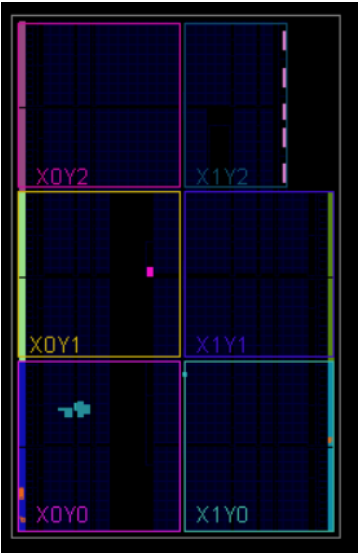
10.3.2. Timing Report summary

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 5.403 ns	Worst Hold Slack (WHS): 0.056 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 78	Total Number of Endpoints: 78	Total Number of Endpoints: 36

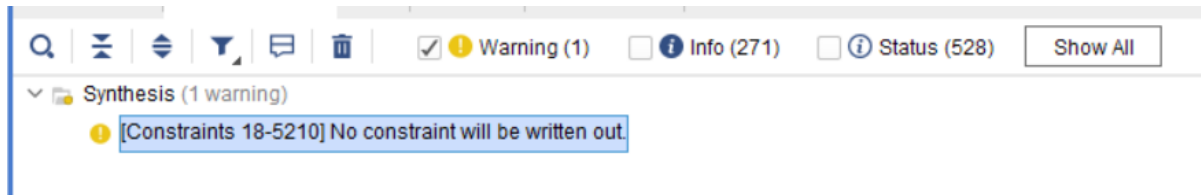
All user specified timing constraints are met.

10.3.3. FPGA device snippet



11. Snippet of the “Messages” tab

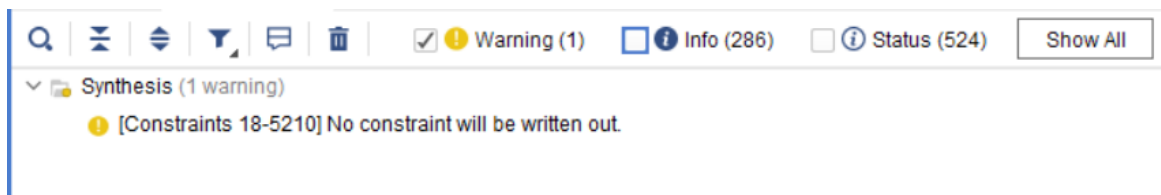
11.1. Gray Encoding



11.2. One_hot Encoding



11.3. Sequential Encoding

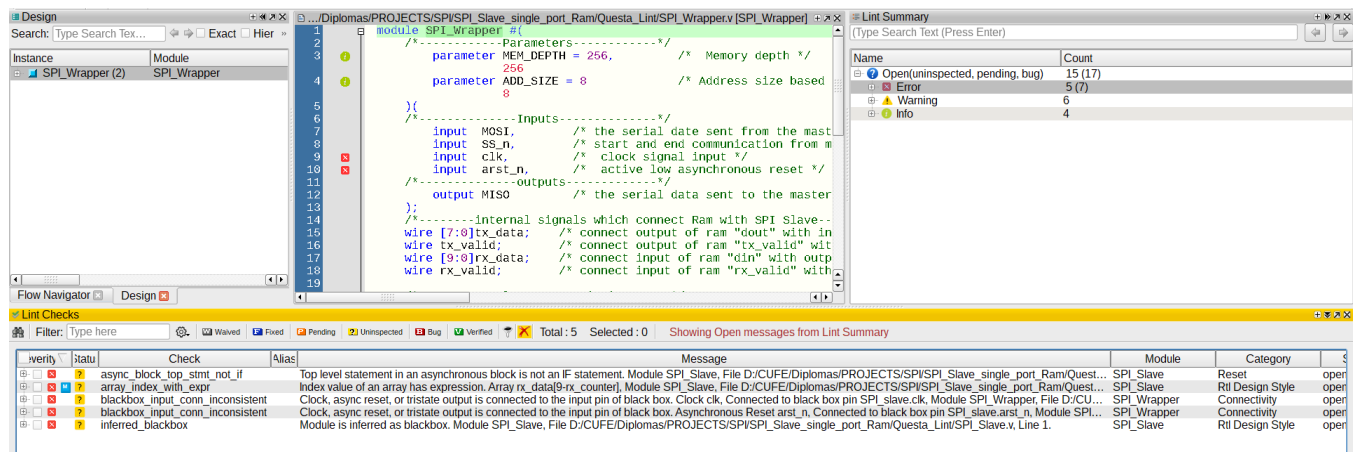


12. RTL Linting (Questa Lint)

To improve RTL code quality and catch structural issues before synthesis, the SPI_Slave, Single_Port_Ram, and SPI_Wrapper modules were checked using Siemens Questa Lint. The tool flags reset, clocking, FSM, and coding-style issues that simulation alone would not necessarily catch.

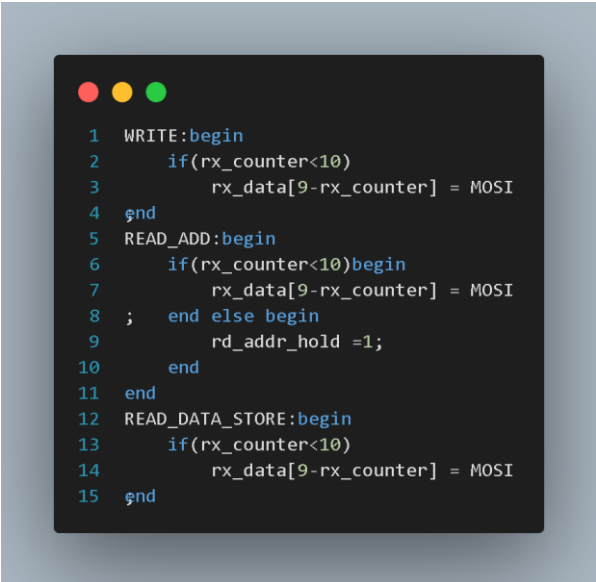
12.1. Initial Lint Run

The first pass over the design surfaced several review items on the SPI_Wrapper hierarchy, including connectivity checks on the (then black-boxed) SPI_Slave instance and RTL design-style checks flagged inside SPI_Slave itself, shown below.



12.2. Bug Fix: Inconsistent Asynchronous Control and Repeated Index Expression

One of the flagged items, `always_has_inconsistent_async_control`, pointed to the `Single_Port_Ram` always block: the `dout` register was only ever driven under the active-low asynchronous reset branch while `addr_internal` was effectively following a different, inconsistent reset treatment inside the same block — a pattern that can cause mismatched reset behavior between related registers. A second, related style issue was an array index built from a repeated inline expression, `rx_data[9-rx_counter]`, used identically in three places in the `SPI_Slave` receive logic.



```
1 WRITE:begin
2   if(rx_counter<10)
3     rx_data[9-rx_counter] = MOSI
4   end
5 READ_ADD:begin
6   if(rx_counter<10)begin
7     rx_data[9-rx_counter] = MOSI
8   ; end else begin
9     rd_addr_hold =1;
10  end
11 end
12 READ_DATA_STORE:begin
13   if(rx_counter<10)
14     rx_data[9-rx_counter] = MOSI
15 end
```

Before: `rx_data[9-rx_counter]` repeated across the `WRITE`, `READ_ADD`, and `READ_DATA_STORE` states

The fix factors the repeated expression into a single continuous assignment, `rx_data_indx`, referenced everywhere the index is needed, and restructures the always block into a clean combinational case statement on the state register (CS) with consistent reset handling for MISO, `rx_data`, and `rd_addr_hold`. This removes the duplicated logic, makes the reset behavior consistent, and makes the code easier to maintain.

```

1 wire rx_data_indx;
2 assign rx_data_indx = 9-rx_counter;
3
4 always @(*) begin
5     if(~arst_n)begin
6         MISO      = 0;
7         rx_data    = 0;
8         rd_addr_hold = 0;
9     end
10    else begin
11        case (CS)
12            IDLE :begin
13                MISO      = 0;
14                rx_data    = 0;
15            end
16            CHX_CMD:begin
17                MISO      = 0;
18                rx_data    = 0;
19            end
20            WRITE:begin
21                if(rx_counter<10)
22                    rx_data[rx_data_indx] = MOS
23            I ;
24            end
25            READ_ADD:begin
26                if(rx_counter<10)begin
27                    rx_data[rx_data_indx] = MOS
28                I ;
29                end else begin
30                    rd_addr_hold =1;
31                end
32            end
33            READ_DATA_STORE:begin
34                if(rx_counter<10)
35                    rx_data[rx_data_indx] = MOS
36            I ;
37            end
38        end
39    end
40 end

```

After: a single rx_data_indx assignment feeding all three states

12.3. Final Lint Summary

After applying the fix, the design was re-checked. The final report shows a Design Quality Score of 99.9%, with zero errors remaining:

Lint Summary	
(Type Search Text (Press Enter))	
Name	Count
Open(uninspected, pending, bug)	4
Warning	1
always_has_inconsistent_async_control	1
Info	3
fsm_without_one_hot_encoding	1
parameter_name_duplicate	2

Final Questa Lint summary – 0 errors after the fix

Severity	Count	Notes
Error	0	No errors remaining after the fix
Warning	1	always_has_inconsistent_async_control – residual note on the RAM always block, reviewed and accepted
Info	3	fsm_without_one_hot_encoding (expected – the sequential/gray/one-hot variants are compared intentionally) and parameter_name_duplicate ×2 (MEM_DEPTH, ADD_SIZE reused by design across SPI_Wrapper and Single_Port_Ram)

Design summary: 36 register bits, 0 latch bits, 0 blackboxes, 0 unresolved modules — confirming the design is fully synchronous and free of unintended latches.

13. ASIC Synthesis and DFT Insertion Flow (Synopsys Design Compiler)

In addition to the Xilinx Vivado FPGA flow, the SPI_Wrapper top module was pushed through a standard-cell ASIC flow using Synopsys Design Compiler, targeting a TSMC 130 nm library (scmetro_tsmc_cl013g_rvt), at the slow-slow 1.08 V / 125°C worst-case corner, with a 100 ns (10 MHz) clock constraint on clk. This step demonstrates the design closing timing and fitting an area/power budget outside the FPGA fabric, and adds Design-for-Test scan structures for manufacturing test.

13.1. Pre-DFT Synthesis

The design was first synthesized without test structures to establish a baseline area, power, and timing result.

Metric	Value
Ports	52
Nets / Cells	11,732 / 11,684
Combinational / Sequential cells	3,432 / 8,250
Total cell area	111,840.63 μm^2
Total power (switching + internal + leakage)	0.349 mW ($6.73 \times 10^{-3} + 0.278 + 0.0646$ mW)
Worst setup slack	MET, +67.81 ns
Worst hold slack	MET, +0.78 ns

The large positive setup slack reflects the low 10 MHz target frequency relative to the achievable speed of the 130 nm library — timing was easily closed with margin to spare.

13.1.1. Setup Timing Report (Pre-DFT)

The worst-case setup path (Design Compiler reports the most critical path first) runs from the MOSI input, through the SPI_Slave receive logic, into the RAM write-data path:

Item	Detail
Startpoint	MOSI (input port, clocked by clk)
Endpoint	RAM/mem_reg_227__2_ (rising-edge flip-flop)
Path group / type	clk / max (setup)
Data arrival time	31.59 ns
Data required time	99.40 ns (100 ns period – 0.20 ns clock uncertainty – 0.40 ns library setup time)
Slack	MET, +67.81 ns

All other setup paths in the design report comparable or larger slack, so the design comfortably meets the 100 ns (10 MHz) clock constraint before any test structures are added.

13.1.2. Hold Timing Report (Pre-DFT)

The worst-case hold path is a short, same-register path with no combinational logic in between, which is typical for the tightest hold margin in a design:

Item	Detail
Startpoint	RAM/tx_valid_reg (rising-edge flip-flop)
Endpoint	RAM/tx_valid_reg (rising-edge flip-flop)

Path group / type	clk / min (hold)
Data arrival time	0.85 ns
Data required time	-0.11 ns (0.10 ns clock uncertainty – 0.21 ns library hold time)
Slack	MET, +0.95 ns

Hold slack is met with comfortable margin, so no hold-fixing buffers were required in the pre-DFT netlist.

13.2. Design-for-Test (Scan) Insertion

Dedicated DFT ports (test_mode, SI, SE, scan_clk, scan_rst, SO) were added and muxed onto the functional clock and reset so the chip can be driven in a test mode after fabrication. Using a full-scan, multiplexed-flip-flop methodology in a single clock domain, DFT Compiler stitched the sequential elements into 21 scan chains of roughly 99 cells each, covering all 2,071 scannable sequential cells with zero DRC violations reported in the post-DFT check.

Metric	Value
Scan chains	21 (full scan, multiplexed flip-flop, no_mix)
Cells per chain / total scan cells	~99 / 2,071 (0 violations)
Ports (incl. test_mode, SI, SE, scan_clk, scan_rst, SO)	144
Total cell area	111,967.72 μm^2 (less than 0.2% area overhead vs. pre-DFT)
Worst setup / hold slack	MET / MET

The pre-DFT DRC check flagged 8 informational warnings about the scan reset feeding data inputs of a few RAM address-register scan cells; these are expected during pre-DFT analysis on cells that will be replaced by scan-equivalent flops and cleared automatically once the chains were stitched, leaving zero post-DFT violations. Scan insertion added negligible area (well under 1%), confirming the design is testable at essentially no silicon cost.

13.2.1. Setup Timing Report (Post-DFT)

After scan insertion, the worst-case setup path shifts slightly (extra scan-mux delay on the data path) but the design still closes with wide margin:

Item	Detail
Startpoint	MOSI (input port, clocked by clk)
Endpoint	RAM/mem_reg_0_2_ (rising-edge flip-flop)
Path group / type	clk / max (setup)
Data arrival time	31.48 ns
Data required time	99.39 ns (100 ns period – 0.20 ns clock uncertainty – 0.41 ns library setup time)
Slack	MET, +67.92 ns

13.2.2. Hold Timing Report (Post-DFT)

The worst-case hold path after scan stitching runs through the new scan-select (SI) mux into the first bit of the RAM memory register:

Item	Detail
Startpoint	RAM/dout_reg_7_ (rising-edge flip-flop)
Endpoint	RAM/mem_reg_0_0_ (rising-edge flip-flop, scan SI input)
Path group / type	clk / min (hold)

Data arrival time	0.62 ns
Data required time	-0.15 ns (0.10 ns clock uncertainty – 0.25 ns library hold time)
Slack	MET, +0.76 ns

Both setup and hold close cleanly post-DFT, confirming the scan chains did not introduce any timing violations. The full DFT rules-check log reported 0 setup and 0 hold violations across all 2,071 scan cells.

13.3. Combined Area and Power Summary

For quick comparison, the table below places the pre-DFT and post-DFT area and power results side by side:

Metric	Pre-DFT (Syn)	Post-DFT
Ports	52	144
Cells	11,684	7,560
Total cell area	111,840.63 μm^2	111,967.72 μm^2 (+0.11%)
Switching power	6.73 $\times 10^{-3}$ mW	6.425 mW
Internal power	0.278 mW	0.334 mW
Leakage power	0.0646 mW	0.0614 mW
Total power	0.349 mW	6.820 mW*
Worst setup slack	MET, +67.81 ns	MET, +67.92 ns
Worst hold slack	MET, +0.95 ns	MET, +0.76 ns

*The jump in reported switching/total power post-DFT is a reporting artifact, not a real functional increase: Design Compiler flagged unannotated switching activity on the newly added scan ports (PWR-414/415) and fell back to default toggle-rate estimates for them at low analysis effort. The RAM and SPI_Slave sub-block power (0.413 mW and 2.89 $\times 10^{-3}$ mW) barely changed from the pre-DFT numbers, confirming the functional logic itself is unaffected — only the unannotated test-port estimate inflates the top-level total.

13.4. Summary

Combined with the Questa Lint results, the SPI_Wrapper design is lint-clean (0 errors), meets timing comfortably at 10 MHz in a 130 nm standard-cell library both before and after scan insertion, and achieves full-scan testability (21 chains, 2,071 cells, 0 DRC violations) for under 0.2% area overhead — complementing the three FPGA encoding implementations (Gray, One-hot, Sequential) already presented in this report.